Resume Text Analytics for Intelligent College Placement Support

A Project Based Learning Report Submitted in partial fulfilment of the requirements for the award of the degree

of

Bachelor of Technology

TEXT ANALYTICS

23SDCS18A

Submitted by

2310080022: JAGGAVARAPU AKHIL SAI REDDY

2310080007: MOHAMMED SHOAIB HANNAN

Under the guidance of

Dr. MANI KUMAR CHAKILAM

Department of Artificial Intelligence and Data science

Koneru Lakshmaiah Education Foundation, Aziz Nagar

Aziz Nagar – 500075

FEB - 2026.

Introduction

Resume Text Analytics for Intelligent College Placement Support is an advanced approach that uses **Natural Language Processing (NLP)** and **Machine Learning** to analyze and interpret student resumes. It focuses on extracting useful information such as skills, education, projects, and experience from unstructured resume text. This helps transform raw data into structured insights that can be used effectively during the placement process.

In traditional college placement systems, recruiters or placement officers manually review a large number of resumes, which is time-consuming and may lead to inconsistencies or bias. Resume text analytics helps overcome these challenges by automating the screening process. It ensures faster evaluation and improves accuracy by consistently applying predefined criteria to all resumes, especially in large institutions where many students participate in placement drives.

The system uses techniques like keyword extraction, entity recognition, and semantic analysis to understand resume content. Based on this, it matches student profiles with job descriptions, ranks candidates, and identifies skill gaps. This allows students to receive personalized feedback and improve their resumes. It also helps placement cells maintain a centralized database for quick access and better management.

Modern resume analytics systems also use advanced technologies such as deep learning and predictive analytics. These systems can predict candidate success, suggest career paths, and provide useful insights to both students and recruiters. Integration with job portals further improves the placement process, making it more efficient and connected.

Another key advantage is promoting fairness and transparency in recruitment. Since evaluation is data-driven, it reduces human bias and ensures equal opportunities. It also helps institutions analyze placement trends and improve training programs based on industry demands.

In this project, a web-based system is developed to make resume analysis interactive and user-friendly. The frontend is built using React, allowing students to upload resumes, view results, and receive feedback through a simple dashboard. This improves usability and user experience.

On the backend, MongoDB is used to store resumes, extracted data, and results. It efficiently handles unstructured data and supports quick access, making it suitable for large-scale student data management.

The system connects frontend and backend using APIs for smooth communication. Uploaded resumes are processed, analyzed, and the results are displayed clearly with skills, scores, and suggestions. The system is also scalable, allowing future improvements like better AI models and job portal integration.

Overall, Resume Text Analytics improves the efficiency and effectiveness of college placement systems. It reduces manual effort, increases placement success, and ensures better matching between students and job opportunities.

Literature Review

The field of Resume Text Analytics has gained significant attention with the advancement of Natural Language Processing (NLP) and Machine Learning. Early research primarily focused on automating resume parsing, where systems extract key information such as skills, education, and experience from unstructured documents. These systems relied on rule-based methods and basic text processing techniques to identify relevant sections in resumes. Studies show that NLP-based resume parsers can efficiently convert resumes into structured formats, enabling easier analysis and candidate filtering. This laid the foundation for modern intelligent recruitment and placement systems.

Recent studies have expanded beyond simple parsing to include intelligent resume screening and candidate ranking. Researchers introduced similarity-based approaches such as cosine similarity and Jaccard similarity to match resumes with job descriptions. These methods compare textual features between resumes and job requirements to compute relevance scores. Such approaches improve hiring efficiency, reduce bias, and ensure fair evaluation by focusing on skill-based competencies. They are particularly useful in college placement systems where large volumes of resumes need to be processed quickly and accurately.

With the growth of artificial intelligence, resume analytics has evolved from traditional techniques to advanced AI-driven models. Modern research incorporates deep learning architectures and transformer-based models like BERT, which provide better contextual understanding of resume content and improve the accuracy of information extraction. Additionally, hybrid systems combining NLP with Optical Character Recognition (OCR) enable processing of resumes in multiple formats, increasing system flexibility and performance.

Recent advancements also emphasize the use of Named Entity Recognition (NER) to identify and categorize entities such as skills, organizations, job roles, and educational qualifications. NER-based systems enhance the precision of information extraction and enable better structuring of resume data. Additionally, ranking algorithms and recommendation systems are integrated to suggest suitable job roles and career paths for candidates based on their profiles.

Overall, the literature indicates that Resume Text Analytics is rapidly evolving toward more intelligent, scalable, and fair systems. The integration of advanced NLP techniques, machine learning models, and real-world applications makes it highly relevant for improving college placement support systems. These developments not only enhance automation but also provide meaningful insights that help students align their skills with industry requirements.

Methodology

I. Data Collection

The process begins with collecting student resumes in formats such as PDF and DOCX through a web interface developed using React. Students upload their resumes through the platform, and these resumes are stored in the backend database for further processing.

II. Resume Parsing and Text Extraction

Once uploaded, resumes are processed to extract textual content. Parsing libraries are used for PDF and DOCX files, and in cases of scanned documents, **Optical Character Recognition (OCR)** techniques are applied. This step converts unstructured resume files into machine-readable text, enabling further analysis.

III. Data Preprocessing

The extracted text is cleaned and prepared by removing special characters, converting text to lowercase, eliminating stop words, and performing tokenization. Libraries such as NLTK or spaCy are used for efficient preprocessing, ensuring that the data is consistent and suitable for analysis.

IV. Feature Extraction

In this stage, important information such as skills, education, experience, and projects is extracted using NLP techniques. Methods like keyword extraction and **Named Entity Recognition (NER)** help identify relevant entities. The extracted information is then structured into meaningful features for further processing.

V. Resume Classification

The system categorizes resumes into different domains such as software, core, or management. Machine learning algorithms from Scikit-learn like Naïve Bayes, Logistic Regression, or Support Vector Machines (SVM) are used for classification. This helps in organizing candidates based on their domain expertise.

VI. Job Matching and Similarity Analysis

The extracted features are compared with job descriptions using similarity techniques such as cosine similarity. A matching score is generated to indicate how well a candidate's profile aligns with a specific job role, improving the accuracy of candidate selection.

VII. Ranking System

Candidates are ranked based on their matching scores, skills, and experience. This ranking helps recruiters and placement officers quickly identify the most suitable candidates for job opportunities.

VIII. Feedback Generation

The system provides personalized feedback by identifying missing or weak skills in the resume. Suggestions are given to help students improve their profiles and align with industry expectations.

IX. Database Management

All processed data, including resumes, extracted features, scores, and results, are stored in MongoDB. This database efficiently handles large volumes of unstructured data and ensures quick retrieval and updates.

X. Frontend–Backend Integration

The frontend (React) and backend are connected using APIs for smooth communication. When a resume is uploaded, it is processed in the backend, analyzed using NLP and machine learning techniques, stored in the database, and the results are sent back to the frontend.

XI. System Output

The final output is displayed through a user-friendly interface and includes extracted skills, job matching scores, candidate rankings, and suggestions for improvement. This ensures that both students and placement officers can easily understand and utilize the results.

References

[1] J. Zhang and Y. Wang, "Resume Information Extraction with Natural Language Processing," in *Proc. IEEE Int. Conf. Big Data*, 2019, pp. 1234–1240.

[2] S. Malherbe and M. Moens, "Automated Resume Classification using Machine Learning Techniques," *Expert Systems with Applications*, vol. 42, no. 21, pp. 7345–7355, 2015.

[3] A. Jain and V. Gupta, "Resume Screening and Ranking using Natural Language Processing and Machine Learning," in *Proc. Int. Conf. Data Mining*, 2020, pp. 98–105.

[4] F. Sultana, A. Sufian, and P. Dutta, "A Review of Text Mining Techniques in Recruitment Systems," in *Intelligent Computing Applications*, Springer, 2020, pp. 1–15.

[5] T. Mikolov et al., "Efficient Estimation of Word Representations in Vector Space," arXiv preprint arXiv:1301.3781, 2013.

[6] J. Devlin, M.-W. Chang, K. Lee, and K. Toutanova, "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," in *Proc. NAACL-HLT*, 2019, pp. 4171–4186.

[7] S. Bird, E. Klein, and E. Loper, *Natural Language Processing with Python*, O'Reilly Media, 2009.

[8] F. Pedregosa et al., "Scikit-learn: Machine Learning in Python," *Journal of Machine Learning Research*, vol. 12, pp. 2825–2830, 2011.

[9] M. Honnibal and I. Montani, "spaCy 2: Natural Language Understanding with Bloom Embeddings," 2017.

[10] G. Salton and C. Buckley, "Term-weighting Approaches in Automatic Text Retrieval," *Information Processing & Management*, vol. 24, no. 5, pp. 513–523, 1988.

[11] R. Baeza-Yates and B. Ribeiro-Neto, *Modern Information Retrieval*, Addison-Wesley, 1999.

[12] K. Sparck Jones, "A Statistical Interpretation of Term Specificity and Its Application in Retrieval," *Journal of Documentation*, vol. 28, no. 1, pp. 11–21, 1972.